

Many-To-Many Relationships : Code-Along (CodeGrade)

 <https://github.com/learn-co-curriculum/python-p4-v2-many-many-relationships>  <https://github.com/learn-co-curriculum/python-p4-v2-many-many-relationships/issues/new>

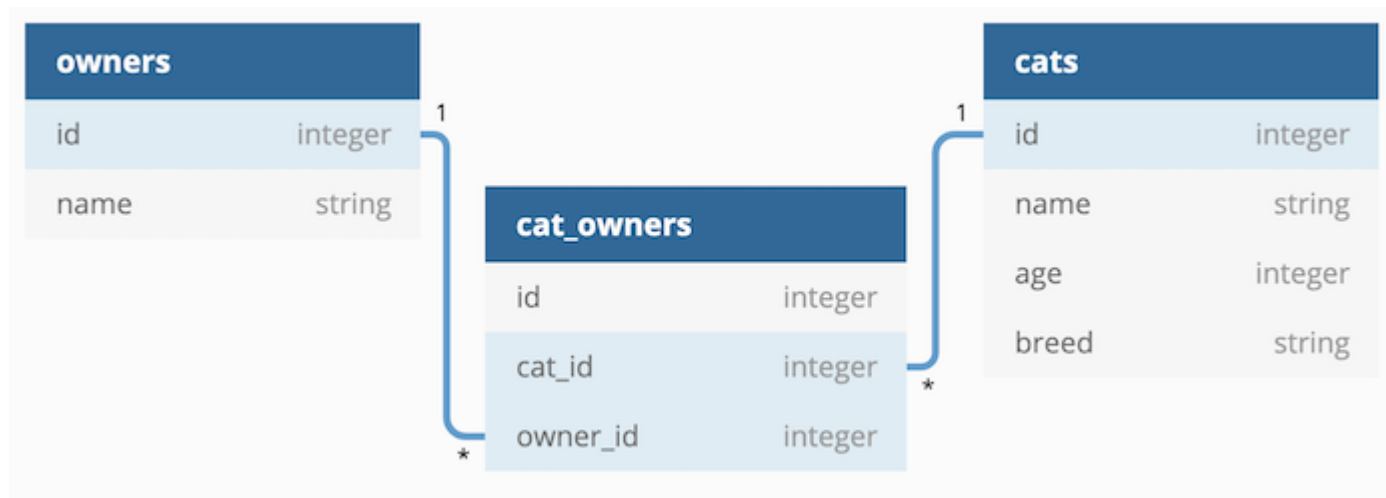
Learning Goals

- Use Flask-SQLAlchemy to join tables with many-to-many relationships.

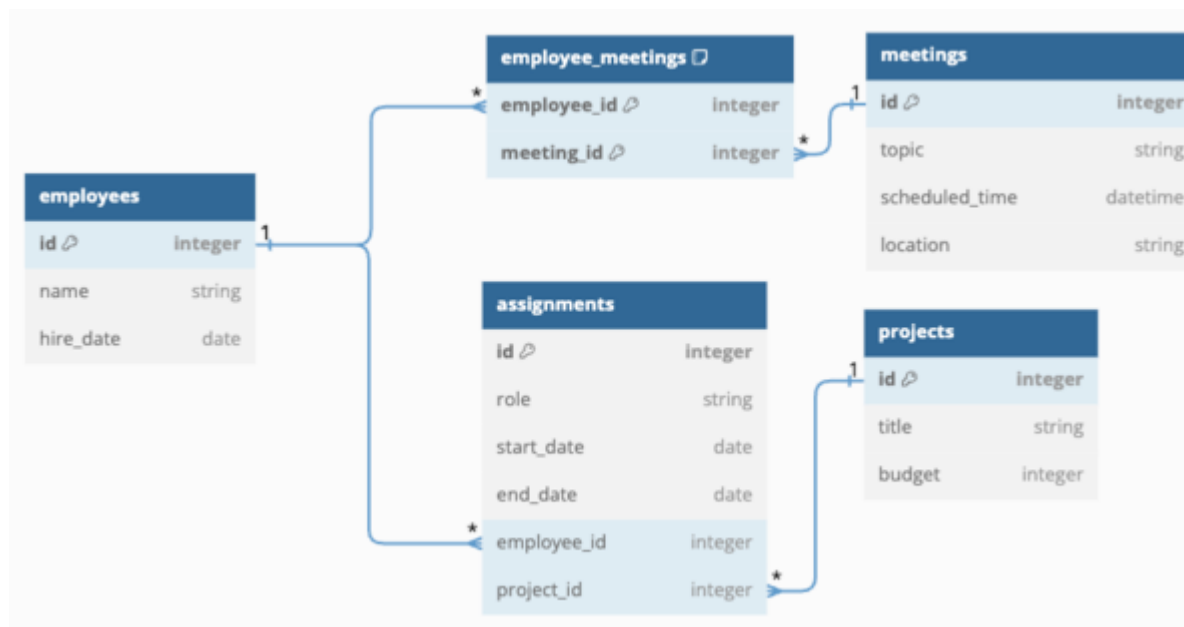
Introduction

In the previous lesson, we saw how to create a **one-to-many** relationship by assigning the correct foreign key on our tables, and using the `relationship()` method along with the `back_populates()` parameter.

In the SQL modules, we learned about one other kind of relationship: the **many-to-many**, also known as the **has many through**, relationship. For instance, in a domain where a cat has many owners and an owner has many cats, we need to create a table named **cat_owners** with foreign keys to the two related tables:



In this lesson, we'll learn different ways to implement **many-to-many** relationships for a data model containing employees, meetings, projects, and assignments:



- An employee **has many** meetings.
- A meeting **has many** employees.
- An employee **has many** projects **through** assignments.

- A project **has many** employees **through** assignments.
 - An assignment **belongs to** an employee.
 - An assignment **belongs to** a project.
-

Setup

This lesson is a code-along, so fork and clone the repo.

Run `pipenv install` to install the dependencies and `pipenv shell` to enter your virtual environment before running your code.

```
$ pipenv install
$ pipenv shell
```

Change into the `server` directory and configure the `FLASK_APP` and `FLASK_RUN_PORT` environment variables:

```
$ cd server
$ export FLASK_APP=app.py
$ export FLASK_RUN_PORT=5555
```

The file `server/models.py` defines models named `Employee`, `Meeting`, and `Project`. Relationships have not been established between the models.

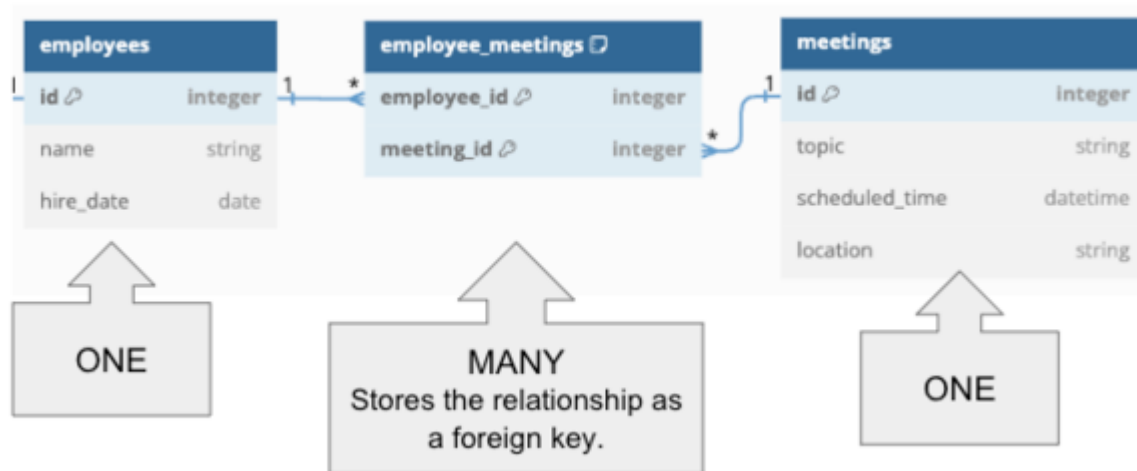
Run the following commands to create and seed the tables with sample data.

```
$ flask db init
$ flask db migrate -m "initial migration"
$ flask db upgrade head
$ python seed.py
```

Many-To-Many with `Table` Objects

Many-to-many relationships in SQLAlchemy use intermediaries called **association tables** (also called join tables). These are tables that exist only to join two related tables together.

There are two approaches to building these associations: association objects, which are most similar to the models we've built so far, and the more common approach, **Table** objects.



We'll implement the many-to-many between **employees** and **meetings** by creating a table named **employee_meetings**. Since **employee_meetings** is on the **many** side of its relationships with **employees** and **meetings**, the table will store two foreign keys **employee_id** and **meeting_id** to reference the primary keys of the other two tables.

Update **models.py** to add the new association table **employee_meetings** as shown below. NOTE: You'll need to place this above the **Employee** and **Meeting** models within **models.py** so they may reference it.

```
# Association table to store many-to-many relationship between employees and meetings
employee_meetings = db.Table(
    'employees_meetings',
    metadata,
    db.Column('employee_id', db.Integer, db.ForeignKey(
        'employees.id'), primary_key=True),
    db.Column('meeting_id', db.Integer, db.ForeignKey(
```

```

        'meetings.id'), primary_key=True)
    )

```

Update `Employee` and `Meeting` to add the relationship between the two models. The `relationship()` method is passed an additional parameter `secondary`, which indicates the relationship is implemented **through** the `employee_meetings` table.

```

class Employee(db.Model):
    __tablename__ = 'employees'

    id = db.Column(db.Integer, primary_key=True)
    name = db.Column(db.String)
    hire_date = db.Column(db.Date)

    # Relationship mapping the employee to related meetings
    meetings = db.relationship(
        'Meeting', secondary=employee_meetings, back_populates='employees')

    def __repr__(self):
        return f'<Employee {self.id}, {self.name}, {self.hire_date}>'

class Meeting(db.Model):
    __tablename__ = 'meetings'

    id = db.Column(db.Integer, primary_key=True)
    topic = db.Column(db.String)
    scheduled_time = db.Column(db.DateTime)
    location = db.Column(db.String)

    # Relationship mapping the meeting to related employees
    employees = db.relationship(
        'Employee', secondary=employee_meetings, back_populates='meetings')

```

```
def __repr__(self):  
    return f'<Meeting {self.id}, {self.topic}, {self.scheduled_time}, {self.location}>'
```

You'll need to migrate the schema with the new table:

```
flask db migrate -m 'add employee_meetings association table'  
flask db upgrade head
```

Update `seed.py` to import the table data structure:

```
from models import db, Employee, Meeting, Project, employee_meetings
```

Update `seed.py` to delete the new table along with the other tables. Since the table is not created from a model class, you'll need to delete it using `db.session.query(employee_meetings).delete()` :

```
# Delete all rows in tables  
db.session.query(employee_meetings).delete()  
db.session.commit()  
Employee.query.delete()  
Meeting.query.delete()  
Project.query.delete()
```

Update `seed.py` to add two meetings to an employee and to add three more employees to a meeting.

```
# Many-to-many relationship between employee and meeting  
  
# Add meetings to an employee  
e1.meetings.append(m1)  
e1.meetings.append(m2)  
# Add employees to a meeting  
m2.employees.append(e2)
```

```
m2.employees.append(e3)
m2.employees.append(e4)
db.session.commit()
```

Re-seed the database:

```
$ python seed.py
```

Let's confirm the database table has the correct rows:

server > instance >  app.db

Search tables...

Tables (6)

> alembic_version

> employees

> meetings

> projects

> assignments

▼ employee_meetings

employee_id

🔑

#

meeting_id

🔑

#

Reset Filters

Records: 5

Search 5 records...

employee_id

🔑

#

🚩

meeting_id

🔑

#

🚩

Search column...

Search column...

1	1	1
2	1	2
3	4	2
4	3	2
5	2	2

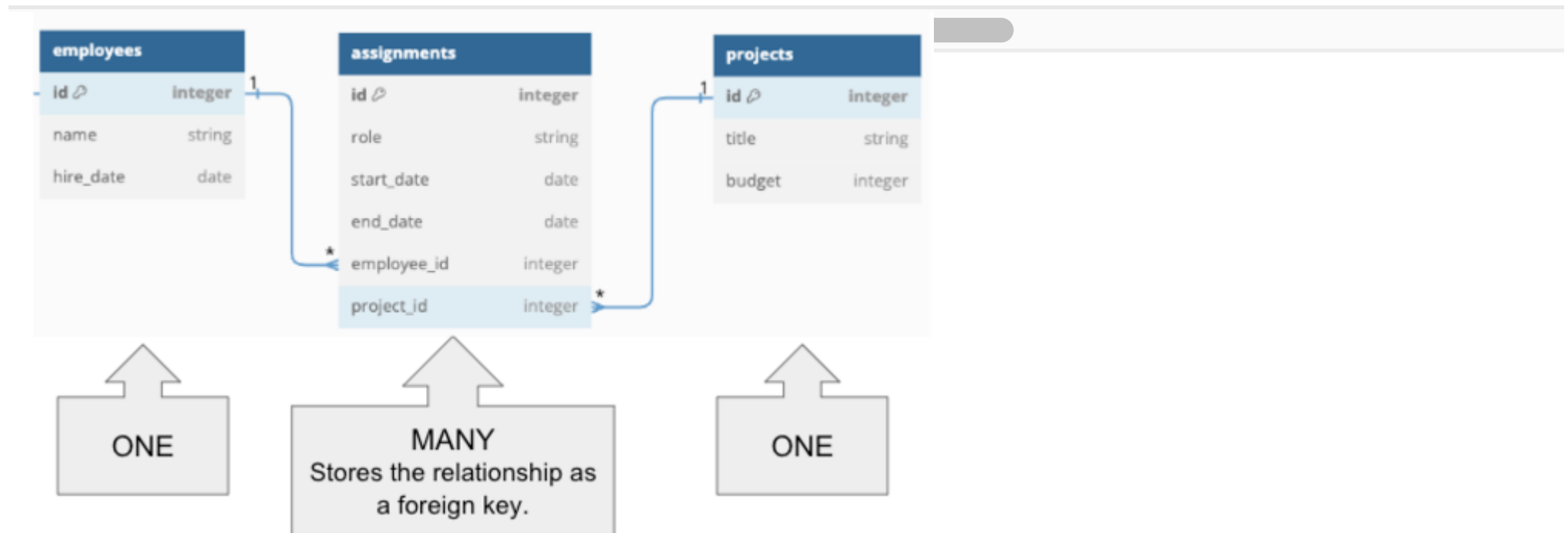
We can explore the relationship using Flask shell:

```
$ flask shell
>>> meeting2 = Meeting.query.filter_by(id = 2).first()
>>> meeting2
<Meeting 2, Github Issues Brainstorming, 2023-12-01 15:15:00, Building D, Room 430>
>>>
>>> meeting2.employees
[<Employee 1, Uri Lee, 2022-05-17>, <Employee 4, Taylor Jai, 2015-01-02>, <Employee 3, Sasha Hao, 2021-12-01>, <Empl
>>>
```

```
>>> employee1 = Employee.query.filter_by(id = 1).first()
>>> employee1.meetings
```

Many-To-Many with Association Objects

An employee is assigned to work on many projects, and a project may have many employees assigned to it. The database needs to keep track of the employee's role on the project, along with the dates they start and end working on the project. We'll use an **association object** to capture the relationship between an employee and a project, along with the attributes (`role` , `start_date` , `end_date`) that are specific to the assignment.



An association object is really just another model, thus the many-to-many relationship between `Employee` and `Project` is implemented as two one-to-many relationships through `Assignment` :

- One-to-many relationship between `Employee` and `Assignment`
- One-to-many relationship between `Project` and `Assignment`

Edit `models.py` to add a new model named `Assignment` having the columns and relationships as shown:

```
# Association Model to store many-to-many relationship between employee and project
class Assignment(db.Model):
    __tablename__ = 'assignments'

    id = db.Column(db.Integer, primary_key=True)
    role = db.Column(db.String)
    start_date = db.Column(db.DateTime)
    end_date = db.Column(db.DateTime)

    # Foreign key to store the employee id
    employee_id = db.Column(db.Integer, db.ForeignKey('employees.id'))
    # Foreign key to store the project id
    project_id = db.Column(db.Integer, db.ForeignKey('projects.id'))

    # Relationship mapping the assignment to related employee
    employee = db.relationship('Employee', back_populates='assignments')
    # Relationship mapping the assignment to related project
    project = db.relationship('Project', back_populates='assignments')

    def __repr__(self):
        return f'<Assignment {self.id}, {self.role}, {self.start_date}, {self.end_date}, {self.employee.name}, {self
```

The `Employee` and `Project` classes must be updated to add the relationship through `Assignment`. We don't use the `secondary` parameter since the relationship is implemented directly with the `Assignment` model rather than an association table.

```
class Employee(db.Model):
    __tablename__ = 'employees'

    id = db.Column(db.Integer, primary_key=True)
```

```
name = db.Column(db.String)
hire_date = db.Column(db.Date)

# Relationship mapping the employee to related meetings
meetings = db.relationship(
    'Meeting', secondary=employee_meetings, back_populates='employees')

# Relationship mapping the employee to related assignments
assignments = db.relationship(
    'Assignment', back_populates='employee', cascade='all, delete-orphan')

def __repr__(self):
    return f'<Employee {self.id}, {self.name}, {self.hire_date}>'

class Project(db.Model):
    __tablename__ = 'projects'

    id = db.Column(db.Integer, primary_key=True)
    title = db.Column(db.String)
    budget = db.Column(db.Integer)

    # Relationship mapping the project to related assignments
    assignments = db.relationship('Assignment', back_populates='project', cascade='all, delete-orphan')

    def __repr__(self):
        return f'<Review {self.id}, {self.title}, {self.budget}>'
```

Migrate the schema to reflect the new model:

```
flask db migrate -m 'add assignment association table'
flask db upgrade head
```

Update `seed.py` to import the new model:

```
from models import db, Employee, Meeting, Project, Assignment, employee_meetings
```

Update `seed.py` to delete the new table:

```
# Delete all rows in tables
db.session.query(employee_meetings).delete()
db.session.commit()
Employee.query.delete()
Meeting.query.delete()
Project.query.delete()
Assignment.query.delete()
```

You'll also need to edit `seed.py` to assign employees to projects. This should go at the end of the file after the employees and projects have been created.

```
# Many-to-many relationship between employee and project through assignment
```

```
a1 = Assignment(role='Project manager',
                start_date=datetime.datetime(2023, 5, 28),
                end_date=datetime.datetime(2023, 10, 30),
                employee=e1,
                project=p1)
a2 = Assignment(role='Flask programmer',
                start_date=datetime.datetime(2023, 6, 10),
                end_date=datetime.datetime(2023, 10, 1),
                employee=e2,
                project=p1)
a3 = Assignment(role='Flask programmer',
                start_date=datetime.datetime(2023, 11, 1),
```

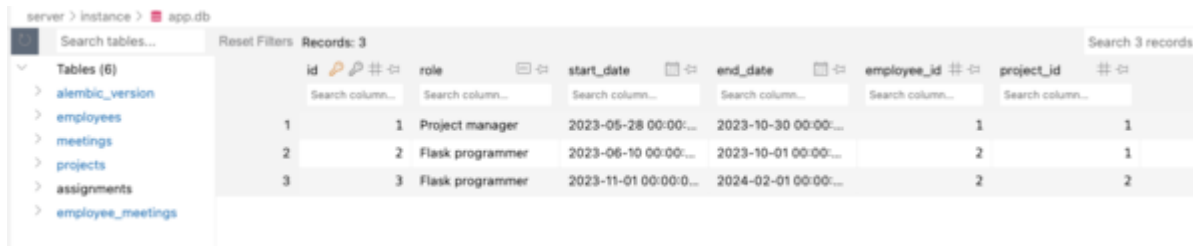
```
end_date=datetime.datetime(2024, 2, 1),
employee=e2,
project=p2)
```

```
db.session.add_all([a1, a2, a3])
db.session.commit()
```

Re-seed the database:

```
$ python seed.py
```

Let's check the **assignments** table to confirm the 3 new rows:



The screenshot shows a database management tool interface. On the left, a sidebar lists tables: alembic_version, employees, meetings, projects, assignments, and employee_meetings. The 'assignments' table is selected, and its records are displayed in a table view. The table has columns: id, role, start_date, end_date, employee_id, and project_id. There are 3 records shown.

id	role	start_date	end_date	employee_id	project_id
1	Project manager	2023-05-28 00:00:00	2023-10-30 00:00:00	1	1
2	Flask programmer	2023-06-10 00:00:00	2023-10-01 00:00:00	2	1
3	Flask programmer	2023-11-01 00:00:00	2024-02-01 00:00:00	2	2

We can use Flask shell to get assignments for an employee or a project:

```
$ flask shell
>>> employee2 = Employee.query.filter_by(id = 2).first()
>>> employee2.assignments
[<Assignment 2, Flask programmer, 2023-06-10 00:00:00, 2023-10-01 00:00:00, Tristan Tal, XYZ Project Flask server>,
>>>
>>> project1 = Project.query.filter_by(id = 1).first()
>>> project1.assignments
[<Assignment 1, Project manager, 2023-05-28 00:00:00, 2023-10-30 00:00:00, Uri Lee, XYZ Project Flask server>, <Assi
```

Association Proxy

What about getting a list of projects for a given employee, or a list of employees for a given project? Currently we would have to do this by iterating over the intermediary assignments. For example, to get a list of employees working on project #1, we need to iterate through the project's assignments to then get the employee associated with each assignment:

```
>>> project1 = Project.query.filter_by(id = 1).first()
>>> [assignment.employee for assignment in project1.assignments]
[<Employee 1, Uri Lee, 2022-05-17>, <Employee 2, Tristan Tal, 2020-01-30>]
>>>
```

SQLAlchemy provides a construct called an **association proxy** (imported from sqlalchemy.ext.associationproxy) that lets a model effectively reach across the intermediary model to access the related model. Let's update `Employee` and `Project` to add an **association proxy** to each model.

```
class Employee(db.Model):
    __tablename__ = 'employees'

    id = db.Column(db.Integer, primary_key=True)
    name = db.Column(db.String)
    hire_date = db.Column(db.Date)

    # Relationship mapping the employee to related meetings
    meetings = db.relationship(
        'Meeting', secondary=employee_meetings, back_populates='employees')

    # Relationship mapping the employee to related assignments
    assignments = db.relationship(
        'Assignment', back_populates='employee', cascade='all, delete-orphan')

    # Association proxy to get projects for this employee through assignments
    projects = association_proxy('assignments', 'project',
```

```
creator=lambda project_obj: Assignment(project=project_obj))
```

```
def __repr__(self):
    return f'<Employee {self.id}, {self.name}, {self.hire_date}>'
```

```
class Project(db.Model):
    __tablename__ = 'projects'

    id = db.Column(db.Integer, primary_key=True)
    title = db.Column(db.String)
    budget = db.Column(db.Integer)

    # Relationship mapping the project to related assignments
    assignments = db.relationship(
        'Assignment', back_populates='project', cascade='all, delete-orphan')

    # Association proxy to get employees for this project through assignments
    employees = association_proxy('assignments', 'employee',
                                  creator=lambda employee_obj: Assignment(employee=employee_obj))

    def __repr__(self):
        return f'<Review {self.id}, {self.title}, {self.budget}>'
```

The first argument indicates the relationship property we intend to use as 'connector' and the second argument indicates the name of the model we're trying to connect to. The `creator` parameter takes a function (an anonymous lambda function in this case) which accepts as argument an object of the other independent class and returns the corresponding object of the 'connecting' class that made the connection possible. For the association proxy inside `Employee`, we're saying that we want to use the `assignments` relationship to connect to the `Project` model. The `creator` function takes a `Project` object and returns an `Assignment` object.

Now we can easily get the employees for a project:

```
>>> # Use association proxy to get employees for a project
>>> project1 = Project.query.filter_by(id = 1).first()
>>> project1.employees
[<Employee 1, Uri Lee, 2022-05-17>, <Employee 2, Tristan Tal, 2020-01-30>]
```

As well as the projects for an employee:

```
>>> # Use association proxy to get projects for an employee
>>> employee1 = Employee.query.filter_by(id = 1).first()
>>> employee1.projects
[<Review 1, XYZ Project Flask server, 50000>]
>>>
```

An association proxy does not modify the schema, it simply provides direct read/write access between `Employee` and `Project` across the intermediary `Assignment`.

Conclusion

The power of SQLAlchemy all boils down to understanding database relationships and making use of the correct classes and methods. By leveraging "convention over configuration", we're able to quickly set up complex associations between multiple models with just a few lines of code.

The one-to-many and many-to-many relationships are the most common when working with relational databases. By understanding the conventions SQLAlchemy expects you to follow, and how the underlying database relationships work, you have the ability to model all kinds of complex, real-world concepts in your code!

Resources

- [SQLAlchemy ORM Documentation](https://docs.sqlalchemy.org/en/14/orm/) ➞ [\(https://docs.sqlalchemy.org/en/14/orm/\)](https://docs.sqlalchemy.org/en/14/orm/)
- [Basic Relationship Patterns - SQLAlchemy](https://docs.sqlalchemy.org/en/14/orm/basic_relationships.html) ➞ [\(https://docs.sqlalchemy.org/en/14/orm/basic_relationships.html\)](https://docs.sqlalchemy.org/en/14/orm/basic_relationships.html)

Solution

```
# server/models.py

from flask_sqlalchemy import SQLAlchemy
from sqlalchemy import MetaData
from sqlalchemy.ext.associationproxy import association_proxy

metadata = MetaData(naming_convention={
    "fk": "fk_%(table_name)s_%(column_0_name)s_%(referred_table_name)s",
})

db = SQLAlchemy(metadata=metadata)

# Association table to store many-to-many relationship between employees and meetings
employee_meetings = db.Table(
    'employee_meetings',
    metadata,
    db.Column('employee_id', db.Integer, db.ForeignKey(
        'employees.id'), primary_key=True),
    db.Column('meeting_id', db.Integer, db.ForeignKey(
        'meetings.id'), primary_key=True)
)

class Employee(db.Model):
    __tablename__ = 'employees'

    id = db.Column(db.Integer, primary_key=True)
    name = db.Column(db.String)
    hire_date = db.Column(db.Date)
```

```
# Relationship mapping the employee to related meetings
meetings = db.relationship(
    'Meeting', secondary=employee_meetings, back_populates='employees')

# Relationship mapping the employee to related assignments
assignments = db.relationship(
    'Assignment', back_populates='employee', cascade='all, delete-orphan')

# Association proxy to get projects for this employee through assignments
projects = association_proxy('assignments', 'project',
                             creator=lambda project_obj: Assignment(project=project_obj))

def __repr__(self):
    return f'<Employee {self.id}, {self.name}, {self.hire_date}>'

class Meeting(db.Model):
    __tablename__ = 'meetings'

    id = db.Column(db.Integer, primary_key=True)
    topic = db.Column(db.String)
    scheduled_time = db.Column(db.DateTime)
    location = db.Column(db.String)

    # Relationship mapping the meeting to related employees
    employees = db.relationship(
        'Employee', secondary=employee_meetings, back_populates='meetings')

    def __repr__(self):
        return f'<Meeting {self.id}, {self.topic}, {self.scheduled_time}, {self.location}>'

class Project(db.Model):
```

```
__tablename__ = 'projects'

id = db.Column(db.Integer, primary_key=True)
title = db.Column(db.String)
budget = db.Column(db.Integer)

# Relationship mapping the project to related assignments
assignments = db.relationship(
    'Assignment', back_populates='project', cascade='all, delete-orphan')

# Association proxy to get employees for this project through assignments
employees = association_proxy('assignments', 'employee',
                              creator=lambda employee_obj: Assignment(employee=employee_obj))

def __repr__(self):
    return f'<Review {self.id}, {self.title}, {self.budget}>'

# Association Model to store many-to-many relationship with attributes between employee and project
class Assignment(db.Model):
    __tablename__ = 'assignments'

    id = db.Column(db.Integer, primary_key=True)
    role = db.Column(db.String)
    start_date = db.Column(db.DateTime)
    end_date = db.Column(db.DateTime)

    # Foreign key to store the employee id
    employee_id = db.Column(db.Integer, db.ForeignKey('employees.id'))
    # Foreign key to store the project id
    project_id = db.Column(db.Integer, db.ForeignKey('projects.id'))

    # Relationship mapping the assignment to related employee
```

```
employee = db.relationship('Employee', back_populates='assignments')
# Relationship mapping the assignment to related project
project = db.relationship('Project', back_populates='assignments')
```

```
def __repr__(self):
```

```
#!/usr/bin/env python3
# server/seed.py
```

```
import datetime
from app import app
from models import db, Employee, Meeting, Project, Assignment, employee_meetings
```

```
with app.app_context():
```

```
# Delete all rows in tables
db.session.query(employee_meetings).delete()
db.session.commit()
Employee.query.delete()
Meeting.query.delete()
Project.query.delete()
Assignment.query.delete()
```

```
# Add employees
```

```
e1 = Employee(name="Uri Lee", hire_date=datetime.datetime(2022, 5, 17))
e2 = Employee(name="Tristan Tal", hire_date=datetime.datetime(2020, 1, 30))
e3 = Employee(name="Sasha Hao", hire_date=datetime.datetime(2021, 12, 1))
e4 = Employee(name="Taylor Jai", hire_date=datetime.datetime(2015, 1, 2))
db.session.add_all([e1, e2, e3, e4])
db.session.commit()
```

```
# Add meetings
```

```
m1 = Meeting(topic="Software Engineering Weekly Update",
             scheduled_time=datetime.datetime(
                 2023, 10, 31, 9, 30),
             location="Building A, Room 142")
m2 = Meeting(topic="Github Issues Brainstorming",
             scheduled_time=datetime.datetime(
                 2023, 12, 1, 15, 15),
             location="Building D, Room 430")
db.session.add_all([m1, m2])
db.session.commit()
```

Add projects

```
p1 = Project(title="XYZ Project Flask server", budget=50000)
p2 = Project(title="XYZ Project React UI", budget=100000)
db.session.add_all([p1, p2])
db.session.commit()
```

Many-to-many relationship between employee and meeting

Add meetings to an employee

```
e1.meetings.append(m1)
e1.meetings.append(m2)
```

Add employees to a meeting

```
m2.employees.append(e2)
m2.employees.append(e3)
m2.employees.append(e4)
```

Many-to-many relationship between employee and project through assignment

```
a1 = Assignment(role='Project manager',
                start_date=datetime.datetime(2023, 5, 28),
                end_date=datetime.datetime(2023, 10, 30),
                employee=e1,
```

```
        project=p1)
a2 = Assignment(role='Flask programmer',
                start_date=datetime.datetime(2023, 6, 10),
                end_date=datetime.datetime(2023, 10, 1),
                employee=e2,
                project=p1)
a3 = Assignment(role='Flask programmer',
                start_date=datetime.datetime(2023, 11, 1),
                end_date=datetime.datetime(2024, 2, 1),
                employee=e2,
                project=p2)

db.session.add_all([a1, a2, a3])
db.session.commit()
```

This tool needs to be loaded in a new browser window

Load Many-To-Many Relationships : Code-Along (CodeGrade) in a new window